

AgentPProf: Operation-Stack Profiling for AI Agent Traces

Abstract

AI-agent executions are no longer well described by one prompt, one session, or one trace tree. A single run can contain prompts, tool calls, GUI actions, browser actions, process events, plans, subagents, safety labels, failure labels, and human subtask boundaries. Existing observability systems usually make one boundary primary, such as a session, span, or prompt tree. Classic profilers make another boundary primary, such as a function call stack. This paper argues that agent profiling needs a smaller abstraction: an operation is a weighted record with fields, and an operation stack is a recursive projection over those fields chosen at query time. Aggregation is not a separate object or a fixed trace tree; it is the result of choosing a stack shape over the same operations. This is not merely a claim about query-time aggregation, which classic profilers and trace processors also support in different forms. The scoped novelty is the combination of an agent-operation record model, recursive multi-field operation-stack projections, and a hidden-label cross-dataset localization benchmark.

We implement this model in the Rust AGENTPPROF profiler and evaluate it through four core research questions, RQ1/E1 through RQ4/E4: three empirical profiling questions plus one systems/reproducibility question rather than a chronological run list. RQ1/E1 tests generality and recursive operation-stack formation: across 15 lightly sampled public labeled sources, AGENTPPROF maps 47,590 samples into the same operation layer, and the same 13,265 operations fold from 9 dataset stacks to 57 phase, 226 semantic, 455 action, or 3,757 fixed-session stacks without changing the input operations. RQ2/E2 is the main hidden-label localization benchmark. On AgentRewardBench, SATraj-OS, AgentNet, and OSWorld-Human, we build six tasks over 34,539 operations and 3,699 positives, score 144 view/ranker policies, and find that task-query operation-stack profiling inspects median 0.0937 top-five work versus 1.0 for flat summaries, improves top-five recall over fixed-session drilldown on 5 of 6 tasks, and reduces median group fragmentation from 285.0 to 157.5 groups. RQ3/E3 isolates mechanisms and profile-configuration actionability. Rank-feature, stack-depth, mapping, transfer, diagnostic-lens, case-packet, and counterfactual analyses show that the useful configuration varies by task and objective: operation-stack views are a Pareto point, not a universal default. Executable profile-spec patches improve AP, top-five lift, and first-positive work on 5 of 6 tasks; a supervised held-out boundary-field ablation improves OSWorld-Human AP from 0.2402 to 0.2583 and reduces groups from 108 to 74, while preserving work counterpoints. RQ4/E4 checks replayability, offline cost, and artifact hygiene: deterministic replay covers 76 tracked profile specs, while source-status, number, rubric, guardrail, and two-abstraction checks remain artifact hygiene rather than empirical profiler accuracy evidence.

The supported claim is therefore a scoped profiling claim: operation-stack profiling can localize, rank, and explain dataset-labeled failures, quality problems, and semantic boundaries in public traces, with less inspection work than flat summaries and a better median-fragmentation tradeoff than the fixed-session drilldown proxy. Fixed-session drilldown is the current trace-tree-shaped baseline; real

OpenTelemetry/OpenInference/Phoenix-style span-tree imports remain future baselines. The paper does not claim human-productivity gains, automatic discovery of all intent boundaries, metric dominance, or complete compatibility with existing trace ecosystems.

1 Introduction

AI-agent traces mix many objects that traditional profiling and tracing systems keep separate. A coding agent may ask a model for a plan, call a shell tool, launch a subagent, inspect a file, and retry a failing command. A web or desktop agent may click, type, observe a page, repeat an action, and receive a benchmark label saying that the action was unsafe, redundant, incorrect, or the start of a human subtask. If the profiler fixes the hierarchy at prompt, session, or span boundaries, these questions become hard to ask: which phases concentrate unsafe operations, which action families cause redundant steps, and which groups of operations cross sessions but share the same failure mode?

AGENTPPROF takes the opposite position. The profiler stores a sequence as operations, where prompts, tool calls, processes, syscalls, GUI actions, and plan steps are operation records or derived fields. Session, span, task, and trace identifiers are fields, exchange containers, or baseline shapes rather than separate profiler objects. Users then choose an operation stack to fold the same operations at different depths. One query may group by task, phase, action; another may group by environment, safety. Mapping and tagging are first-class field-derivation steps, but they do not add a third profiler object. They only write fields before the stack query runs. Thus the novelty is the abstraction that connects stack shape to aggregation: changing the stack changes what is comparable, localizable, and actionable without changing the underlying trace object model.

This paper makes a profiling claim, not a human-productivity claim. A top-tier profiling paper must show that the profiler attributes, localizes, and ranks problems against explicit workload labels, and that the output leads to optimization insight with better label-scored ranking or inspection cost than baselines. For agents, the claim is:

Operation/operation-stack profiling provides label-scored localization and ranking for task-relevant failures, quality problems, and dataset-provided semantic boundaries in real labeled agent traces, while requiring less inspection work than flat summaries and improving the median fragmentation tradeoff relative to fixed-session drilldown.

We evaluate this claim with public labeled traces rather than a human study. A human or agent analyst study would be useful for productivity claims, but it is not required to test label-scored profiler fidelity. Our evaluation therefore uses hidden labels as the oracle for ranked profile groups and is organized around four core research questions. RQ1/E1 tests operation generality, recursive stack folding, field derivation, and human-boundary alignment under the same two-object model. RQ2/E2 is the main hidden-label localization and ranking benchmark. RQ3/E3 isolates mechanisms

and actionability through ranker, stack, mapping, transfer, case, and profile-spec patch ablations. RQ4/E4 checks replayability, offline cost, and claim hygiene; it is not treated as a fourth hidden-label accuracy result. Individual R-runs are provenance for these experiments, not the paper-facing structure. The first three RQs are the three empirical profiling questions; the fourth is a systems/reproducibility question.

2 Design

2.1 Operations

An operation is the only sample-level object in AGENTPPROF. It is a weighted record with string fields. The same representation can encode a prompt, a tool call, an API call, a browser action, a PyAutoGUI action, a process event, or a derived label such as `looping=yes` or `step_correct=incorrect`. External datasets enter the system as operation JSONL. Local Codex or Claude sessions can also be exported as an agent-session trace, imported back, or converted to operation JSONL before profiling.

2.2 Operation Stacks

An operation stack is an ordered list of fields. Folding operations by that list yields the usual folded-stack representation, but the stack is chosen at query time. The same operation file can be folded as a flat task summary, a fixed-session tree, a dataset-native hierarchy, a raw action stack, a semantic operation stack, or an oracle label-drilldown. Fixed sessions and spans are therefore baselines and exchange containers, not core profiler abstractions.

2.3 Mapping and Tagging

Mapping and tagging derive new operation fields before folding. For example, desktop actions such as type and key can map to `phase=input`, while tool/API rows can map to a tool phase before generic action rules run. The Rust CLI exposes inline rules, rule files, profile specs, operation predicates, and stack overrides. —where predicates run after mapping and before stack construction, implementing the query predicate without adding a profiler object. Profile specs record operation files, mappings, predicates, views, stacks, and outputs for reproducibility, while command-line flags can still override the stack query.

3 Evaluation Setup

Table 1 summarizes the public trace families. We do not create new test sets, sync complete image corpora, or depend on gated data. The 15 sources establish generality, while the main hidden-label ranking results use the four oracle-rich families with failure, safety, quality, and human-boundary labels.

For R320, we build six tasks: AgentRewardBench looping, AgentRewardBench side-effect, SATraj unsafe operation, AgentNet incorrect step, AgentNet redundant step, and OSWorld-Human group start. Each task has an oracle-positive field, but non-oracle rankers never read that field. We compare six views: flat, fixed-session, dataset-native hierarchy, raw action stack, semantic operation stack, and label-drilldown. We compare four rankers: width, visible-risk, query-aware, and oracle upper bound. Label-drilldown and oracle upper bound are marked as hidden upper bounds.

The metrics match a profiler paper’s main claim. Fidelity uses `precision@k`, `recall@operation-budget`, `F1@k`, average precision as a block-level AUPRC-style score, `nDCG` over positive operation counts, and `work-to-first-positive`. Fragmentation uses group count, positive group count, and work or groups to reach 50% positive recall. Actionability uses per-task comparisons among stack fields, mapping choices, ranker choices, and oracle headroom. Reproducibility uses repeated Rust `agentpprof -profile-spec` executions over tracked specs and tracked operation JSONL inputs, checking semantic output hashes, sample counts, unique-stack counts, runtime, and output size. Build time is excluded from the profiler runtime measurement.

Table 2 gives the paper-facing evaluation structure. R-numbered runs are provenance, ablations, counterpoints, or audit gates rather than the paper’s main organization. R360 regenerates this four-row result table from tracked artifacts and emits JSON, CSV, Markdown, HTML, and a LaTeX table fragment. It is a table-provenance gate, not a new empirical result or profiler rerun.

The rest of the evaluation follows the role map in Table 3. Each RQ has one primary anchor; smaller R-numbered artifacts appear only as supporting evidence, presentation artifacts, counterpoints, or hygiene gates inside those four RQs. This keeps the paper organized as three empirical profiling experiments plus one systems/reproducibility experiment rather than a chronological run ledger. The role map reads existing tracked artifacts and paper text; it does not fetch, sync, or create data, relabel traces, or rerun the profiler.

4 Results

4.1 RQ1/E1: Generality and Recursive Folding

Evidence contract: claim, the operation/operation stack model covers heterogeneous traces without prompt/session/tool-bound units; oracle, public labels and native fields, especially OSWorld-Human boundaries and AgentNet quality; baseline, dataset-native/no-map/fixed-session stacks plus replay and trace round trips; metric, coverage, stack range, compression, boundary F1/V-measure, and equality; counterpoint, configurable operation stack folding, not complete compatibility or latent-intent recovery. Prompt, session, tool, process, and syscall records are operation fields, not new profiler objects. Claim-test: RQ1/E1 asks whether one operation layer can be folded recursively into task, phase, action, boundary, and fixed-session-shaped stacks by changing mappings and stack fields, without changing the input records or adding a third profiler object.

The 14 core public sources produce 42,590 operations. Adding the supplemental ScaleCUA stream brings the smoke set to 47,590 operations. These operations cover web, mobile, desktop, software, API, tool-agent-user dialogue, expert failure labels, safety labels, human grouped-action labels, and desktop step-quality labels. No source required a prompt-specific, GUI-specific, or safety-specific profiler object.

Recursive stack-depth sweep. On the same 13,265 operations, a depth sweep folds to 9 dataset stacks, 57 phase stacks, 226 tool/semantic stacks, 455 action stacks, and 3,757 fixed-session stacks. A fixed session boundary therefore increases stack count by 417.4× relative to dataset depth. On AgentNet, a tracked profile spec folds 16,741

Table 1: Public labeled trace families used by the evaluation. The main R320 hidden-label profiler benchmark uses the four oracle-rich rows in the bottom half.

Family	Native labels	Access path	Evaluation role
WebLINX, Mind2Web, AgentTrek, WebShop	Web actions, demos, tasks, rewards, action history	Hugging Face viewer or public shards	Web navigation coverage
API-Bank, ToolBench, tau-bench	API calls, tool messages, solution paths, outcomes	Hugging Face rows or repo JSONL	Tool/API and dialogue coverage
SWE-agent	Issue trajectory, command actions, target outcome	Hugging Face viewer	Software-agent coverage
AndroidControl, Odyssey, ScaleCUA	Mobile or GUI actions, instructions, history context	Public/HF samples	Mobile and GUI coverage
AgentRewardBench	Expert success, side-effect, looping, optimality	HF annotations plus cleaned BrowserGym steps	Failure and looping tasks
SATraj-OS	Desktop actions, safety, reward, attack type	HF safety config	Safety task
OSWorld-Human	Human single-action and grouped-action traces	Public GitHub JSON	Human-boundary task
AgentNet	Human desktop PyAutoGUI, step correctness, redundancy	HF repo JSONL streaming	Step-quality tasks

Table 2: Four core research questions: three empirical profiling RQs plus one systems/reproducibility RQ. The table is regenerated from tracked artifacts; R-numbered runs remain provenance and counterpoint records.

RQ / core experiment	Workload	Headline evidence	Scoped conclusion
RQ1/E1: generality and recursive folding	15 public labeled trace families / 47,590 operations, plus local-session and standard-trace exchange fixtures	The same 13,265 operations fold across eight depths, from 9 to 3,757 stacks; leave-dataset-out mapping reduces stacks on 6/9 held-out datasets; 12/12 profile specs remain prompt/session-free; standard-trace exchange round-trips 512 real operations with 512 samples and 11 stacks preserved; boundary backends beat the best simple baseline on 4/5 field-derivation rows.	Two abstractions cover heterogeneous traces; trace containers and mappings are not profiler objects, and field derivation is not automatic intent boundary discovery.
RQ2/E2: hidden-label localization and ranking	Six oracle-backed tasks over AgentRewardBench, SATraj-OS, AgentNet, and OSWorld-Human	The benchmark scores 144 view/ranker policies over 6 tasks, 4 datasets, 34,539 operations, and 3,699 positives. Operation-stack query-aware profiling uses 0.0937 top-five work versus flat 1.0, beats fixed-session top-five recall on 5/6 tasks, and uses 157.5 median groups versus fixed-session 285.	Supported as a hidden-label profiler benchmark with baseline tradeoffs, not human utility.
RQ3/E3: mechanism and actionability	The same six tasks plus held-out OSWorld-Human boundary-backend operations	Executable profile-spec patches improve 5/6 tasks, with median AP delta 0.0376 and top-five lift delta 0.5750. Boundary-derived fields improve OSWorld-Human boundary AP to 0.2583 versus 0.2402 and reduce groups to 74 versus 108, while preserving top-five-work and first-positive-work counterpoints. Rank-feature ablations identify 7 critical and 3 misleading feature rows.	The profiler exposes stack, field, ranker, and profile-spec knobs; it is not an automatic selector or boundary detector.
RQ4/E4: replayability, offline cost, and artifact hygiene	76 tracked profile specs over tracked operation JSONL inputs	Deterministic replay gives 76/76 semantic and 76/76 raw-byte deterministic specs over 152 invocations, with median runtime 1.601s and p95 2.767s.	The offline profiling artifact is replayable and cheap enough for artifact evaluation; this is systems/reproducibility evidence, not a hidden-label accuracy result, live capture overhead result, human productivity result, or trace-ecosystem compatibility result.

operations into 608 diagnostic stacks, while a CLI stack override changes the output to 83 stacks without changing the operation input. The result supports query-time recursive stack choice rather than a fixed prompt/session hierarchy.

The implementation probes keep the same claim boundary. R321 adds query predicates: profile specs over the tracked R300 operation file select 729/729, 714/714, and 4,285/4,285 expected operations with mapping-derived where_rules before folding. R342 checks that 12/12 profile specs compose operation sources, predicates, visible ranking rules, rank mode, and explicit stack depth without introducing prompt/session frames. R353 shows the same operation-file prefix can round-trip through a standard trace container and still fold to the same 512 samples and 11 stacks. These are E1 evidence because they test whether the operation layer stays fixed while views change; the localization and ranker gains from those specs are evaluated under E2/E3.

Field derivation and boundary probes. Held-out mapping rules improve compression from 14.049 to 19.091 on 3,990 operations. Leave-dataset-out mapping reduces unique stacks on 6 of 9 held-out datasets with no negative stack-reduction regressions after API/tool phase precedence is fixed. A supervised OSWorld-Human boundary backend reaches F1 0.7735 on held-out adjacent pairs and writes predicted boundary fields that the Rust profiler folds as ordinary operation fields. Boundary backends therefore extend field derivation, not the profiler object model. The field-derivation audit keeps both support and counterpoints visible: held-out mapping improves compression but coarsens action-to-phase V-measure from 0.9343 to 0.7416; profile-spec composition stays prompt/session-free on 12/12 variants; and learned boundary backends beat the best simple baseline on 4/5 rows, while AgentRewardBench looping is fully explained by repeat_signal_change. This supports first-class field derivation and suitability checks, not automatic intent boundary

Table 3: R374 core-experiment role map. Each row separates the primary anchor from support/presentation roles and the non-claim boundary; R374 is a paper organization gate, not a profiler run.

Core experiment	Primary anchor	Support / presentation roles	Non-claim boundary
RQ1/E1: generality and recursive folding	One operation layer over 47,590 operations, recursive stack-depth sweep, profile-spec override, standard-trace round trip, and field derivation (sources: R286/R290/R342/R353/R366).	Dataset coverage, deterministic mapping, leave-dataset-out mapping, and supervised boundary backend probes (sources: R279-R285/R297/R299). Paper-structure gates keep RQ1 as abstraction evidence rather than ranker/actionability evidence (sources: R359-R364/R367/R370-R372).	Not complete trace-ecosystem compatibility and not automatic discovery of every latent intent boundary.
RQ2/E2: hidden-label localization and ranking	Hidden-label localization benchmark over six real labeled tasks, 34,539 operations, 3,699 positives, and 144 policies, scored only after profiling (source: R320).	Uncertainty, negative control, work budgets, fragmentation, sequence scope, metric surface, oracle depth, and trace-tree-shaped baseline scope (sources: R330/R331/R333/R334/R337/R339/R344/R355/R368). Scored outputs become tradeoff plots, headline rows, case cards, and task-level claim verdicts (sources: R363/R365/R373).	Not metric dominance, not human or agent analyst productivity, and not superiority over imported ecosystem traces.
RQ3/E3: mechanism and actionability	Rank-feature mechanisms, feature ablations, executable profile-spec patches, boundary-field repair, and field-derivation mechanism audit (sources: R324/R325/R354/R358/R366).	View/ranker actionability, transfer, mechanism attribution, diagnostic lenses, case packets, baseline contrast, and action counterfactuals (sources: R335/R340/R341/R345-R350). Actionability knobs and task verdicts expose configuration guidance without making a new profiler object or experiment (sources: R363/R365/R373).	Not an automatic patch selector, label-free universal selector, or automatic boundary detector.
RQ4/E4: replayability, offline cost, and artifact hygiene	Profile-spec replay over 76 tracked specs executed twice, 152 invocations, deterministic semantic/raw-byte outputs, median 1.601s and p95 2.767s per spec (sources: R327/R328).	Source provenance, number alignment, rubric coverage, reviewer gates, and paper-organization hygiene (sources: R338/R352/R356/R357/R359-R374). Optional future human-study protocol artifacts are not used for the main claim (sources: R315/R316).	Not hidden-label accuracy evidence, not live eBPF overhead, not human utility, and not complete ecosystem compatibility.

discovery and not a new profiler object. Rank-feature criticality from the same audit belongs to RQ3/E3's mechanism/actionability evidence, not to RQ1/E1's abstraction claim.

Human-boundary oracle. OSWorld-Human gives the strongest direct boundary oracle. On 320 exact-aligned tasks with 4,011 operations and 2,075 human groups, grouped-depth stacks reduce 482 action stacks to 106 grouped stacks. A conservative group_pattern projection has human-group boundary F1 0.627 with precision 1.0 and recall 0.456. This is not complete intent discovery, but it shows that the same single-action sequence can be folded at action depth or human-group depth and scored against a human boundary oracle.

4.2 RQ2/E2: Hidden-Label Localization and Ranking

Evidence contract: claim, operation-stack profiling localizes and ranks real labeled failures, quality problems, safety issues, and semantic boundaries; oracle, 3,699 positives over 34,539 operations; baseline, flat, fixed-session drilldown, dataset-native, raw-action, operation-stack, label-drilldown, and oracle upper bound; metric, precision@k, recall, F1, AUPRC-style AP, nDCG, budget recall/F1, work-to-first-positive, and groups; counterpoint, flat and dataset-native views can win broad-recall or nDCG goals, so the claim is Pareto tradeoff rather than metric dominance. Claim-test: RQ2/E2 asks whether hot stacks and top-ranked groups correspond to hidden positives while requiring less inspection work than flat summaries and improving the localization-budget/fragmentation tradeoff relative to fixed-session drilldown.

Table 4 gives the main R320 accuracy result. Flat summaries recover all positives, but only by inspecting the whole task, so their median work@5 is 1.0 and their work-to-first-positive is 1.0. Fixed-session drilldown finds an early positive with median work-to-first-positive 0.0044 and median work@5 0.0163, but its median recall@5 is only 0.0233 and it fragments the workload into 285.0 groups. Query-aware operation stacks inspect median 0.0937 work in the top five groups, reach median recall@budget30 of 0.3900, and reduce the median group count to 157.5. R368 makes this baseline scope explicit. It treats fixed-session drilldown as the evaluated

trace-tree-shaped baseline, not as evidence of full OpenTelemetry/Phoenix/LangSmith/Perfetto compatibility. On the same R320 rows, operation-stack query-aware improves fixed-session top-five recall and F1 on 5/6 tasks, budget-30 recall on 4/6, and group count on 4/6, while preserving the fixed-session counterpoint: top-five work and first-positive work are better for fixed-session on 4/6 tasks. R355 gives the depth-aware version of the same claim: operation-stack rankings beat fixed-session on budget-30 positive-unit recall for 20/24 task-depth rows and use fewer groups to reach 50% positive units on 22/24 rows. Thus the claim is a better fragmentation tradeoff at comparable localization budgets, not dominance over every trace-tree-shaped objective.

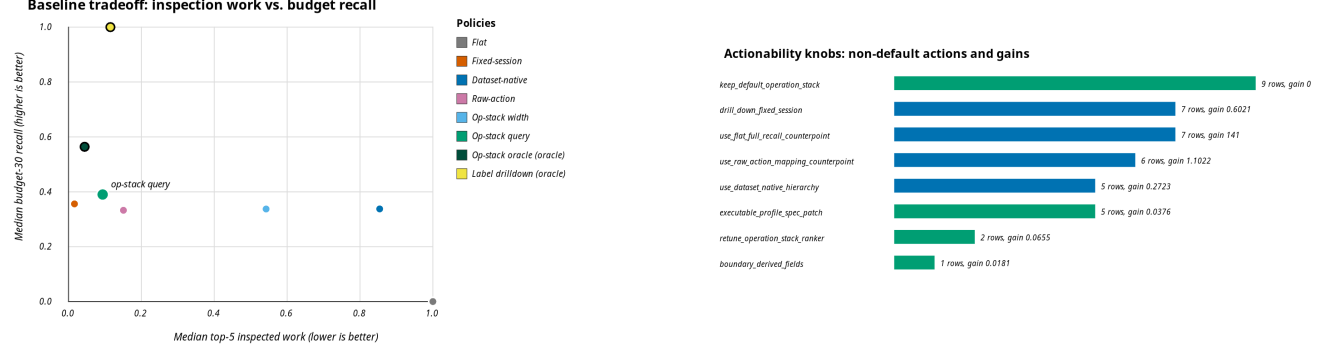
R363 packages the same scored outputs into five paper-facing views instead of a flamegraph-only artifact. Table 5 gives the compact version used by the paper; the artifact also emits SVG, CSV, HTML, JSON, and the table fragment from tracked R320/R345/R348/R354/R355/R358 inputs. The views are not new profiler objects or a fifth experiment. They are different ways to inspect operation-stack outputs and their baseline counterpoints.

R365 further distills the same E2/E3 evidence into headline rows and task case cards for the main paper, instead of exposing the chronological run history. It only reads tracked artifacts and does not download data, relabel traces, or rerun the profiler. R365 is therefore a paper-integration artifact, not a fifth experiment and not a new profiler object beyond operations and operation stacks. Table 6 gives the compact main-paper version. Table 7 expands the six generated task cards: each row keeps the best visible policy, operation-stack top-five work/recall/lift, profile-spec patch verdict, optimization action, and counterpoint. The purpose is to make the actionability claim concrete at task level: the profiler does not only rank hot groups, but also names whether to change stack fields, rankers, mappings, profile specs, or to keep a flat/fixed-session/raw-action counterpoint view.

R373 converts those task cards into a claim-verdict matrix rather than another run list. Table 8 reads the tracked R320/R354/R355/R358/R365 artifacts and asks, for each labeled task, which part of the main profiling claim is supported and which baseline remains the counterpoint. All six tasks improve AP and top-five inspected work over flat summaries; five improve top-five recall over fixed-session drilldown; four reduce fixed-session group fragmentation; and all

Table 4: R320 hidden-label profiler benchmark. Hidden policies read oracle labels and are included only as upper bounds. WTFP is work-to-first-positive.

Policy	Hidden	AP	nDCG	P@5	R@5	F1@5	Work@5	R@30%	WTFP
flat:width	0	0.1678	1.0000	0.1678	1.0000	0.2868	1.0000	0.0000	1.0000
fixed-session:query-aware	0	0.3476	0.7642	0.2555	0.0233	0.0427	0.0163	0.3559	0.0044
dataset-native:query-aware	0	0.3572	0.7445	0.1712	0.8665	0.2822	0.8539	0.3377	0.0582
raw-action:query-aware	0	0.2744	0.5012	0.2513	0.2442	0.2195	0.1507	0.3325	0.0374
operation-stack:width	0	0.1610	0.9364	0.1398	0.4746	0.2147	0.5424	0.3372	0.1694
operation-stack:query-aware	0	0.3117	0.5487	0.1991	0.1880	0.1981	0.0937	0.3900	0.0378
operation-stack:oracle-upper	1	0.5993	0.6372	0.8984	0.3004	0.4029	0.0441	0.5640	0.0122
label-drilldown:oracle-upper	1	1.0000	1.0000	1.0000	0.6703	0.7972	0.1150	1.0000	0.0377

**Figure 1: Two R363 non-flamegraph paper views. Left: the RQ2/E2 baseline tradeoff places operation-stack query-aware profiling in the low-inspection-work region while preserving fixed-session and oracle counterpoints. Right: the RQ3/E3 actionability view turns the same scored outputs into profile-configuration knobs, showing that many objectives require changing the view, ranker, profile spec, or boundary-derived fields rather than using one fixed default view.****Table 5: R363 paper-facing visualization portfolio. The views are generated from tracked artifacts and summarize E2/E3 tradeoffs; R363 is not a new empirical result.**

View	Artifact	Paper-facing evidence	Role
Baseline tradeoff	baseline-tradeoff.svg	Op-stack query Work@5 0.0937 vs flat 1.0; R@30% 0.3900 vs fixed-session 0.3559; groups 157.5 vs 285.0.	E2
Metric heatmap	metric-heatmap.svg	Shows AP 0.3117, R@30% 0.3900, Work@5 0.0937, and the fixed-session WTFP counterpoint 0.0044.	E2
Diagnostic lenses	diagnostic-lenses.svg	Six lenses over 36 objective rows: operation-stack-family views win 11/36, while baseline counterpoints win 25/36.	E3
Actionability knobs	actionability-knobs.svg	27/36 objective rows require non-default actions; R354 accepts 5/6 profile-spec patches; R358 boundary fields add AP 0.0181.	E3
Oracle-depth adequacy	oracle-depth-adequacy.svg	Lower top-5 unit work than flat on 24/24 rows; higher fixed-session unit recall on 20/24; fewer groups to 50% positives on 22/24.	E2/E3

Table 6: R365 E2/E3 headline and case-study selector. It reads tracked artifacts only, does not download data or rerun the profiler, and is not a fifth experiment.

Row	Role	Main numbers	Counterpoint
H1	localization/work	6 tasks / 4 datasets / 34,539 operations / 3,699 positives; Work@5 0.0937 vs flat 1.0; R@30% 0.3900 vs fixed-session 0.3559.	Fixed-session WTFP 0.0044; dataset-native R@5 0.8665 at high work.
H2	fragmentation	Groups 157.5 vs fixed-session 285; top-five recall wins 5/6; 30% budget inspects fewer groups on 5/6 tasks.	Fixed-session remains an early-hit counterpoint.
H3	oracle depth	24/24 rows have lower top-five unit work than flat; 20/24 beat fixed-session unit recall; 22/24 use fewer groups to 50% positives.	Session labels and positive-run proxies are not human intent labels.
H4	actionability	27/36 objective rows need non-default actions; 25/36 require view changes; R354 accepts 5/6 patches with median AP delta 0.0376 and lift delta 0.5750.	Not an automatic patch selector or human-productivity result.
H5	boundary mechanism	Learned-boundary AP 0.2583 vs semantic-width 0.2402; groups 74 vs 108; top-five recall delta +0.1111.	Top-five work and first-positive work worsen, so this is a scoped repair.

six have an actionable configuration result, either an accepted R354 profile patch or the R358 boundary-field repair. The same

table keeps the negative evidence visible: fixed-session remains the first-positive or group-count counterpoint on several tasks, and

Table 7: R365 task-level case cards over the six real labeled tasks. The table is extracted from tracked R320/R345/R348/R354 artifacts; hidden labels score the rows after profiling but are not read by visible policies or optimization actions.

Task / oracle	Best visible policy	Op-stack top-5	Actionable knob	Counterpoint
agentreward_looping (looping)	(failure- dataset_native:query_aware	work 0.4938; recall 0.6508; lift 1.3179	Keep repeat_signal in the stack, but add prevalence-aware ranking because looping positives are common. Patch: accept_patch (0.1155).	top5_recall->flat:width; first_positive->raw_action_stack:query_aware
agentreward_side_effect (failure-side-effect)	fixed_session:width	work 0.1454; recall 0.1139; lift 0.7831	Increase weight on write/input actions or use a deeper side-effect mapping before ranking. Patch: accept_patch (0.0591).	top5_recall->flat:width; top5_lift->raw_action_stack:query_aware
satraj_unsafe (safety)	operation_stack:query_aware	work 0.042; recall 0.2621; lift 6.2384	Use environment + phase + action stack fields; prioritize risky environments and write actions. Patch: accept_patch (0.5081).	top5_recall->flat:width; first_positive->fixed_session:query_aware
agentnet_incorrect_step (step-quality)	raw_action_stack:width	work 0.0014; recall 0.0034; lift 2.4057	Use desktop environment + phase + repeat/action fields, then drill into fixed sessions for examples. Patch: accept_patch (0.016).	top5_recall->flat:width
agentnet_redundant_step (step-quality)	raw_action_stack:width	work 0.0089; recall 0.0177; lift 1.9838	Use desktop environment + phase + repeat/action fields, then drill into fixed sessions for examples. Patch: accept_patch (0.0011).	top5_recall->flat:width; first_positive->fixed_session:query_aware
osworld_group_start (human-boundary)	flat:width	work 0.4074; recall 0.3874; lift 0.951	Use group-depth or boundary-derived fields for higher recall; action-depth alone fragments starts. Patch: reject_patch_or_needs_new_mapping (-0.0004).	top5_recall->flat:width; top5_lift->fixed_session:query_aware; first_positive->fixed_session:query_aware

OSWorld-Human needs boundary-derived fields rather than visible phase/action ranking alone.

The E2 result is deliberately a Pareto claim, not universal dominance. Operation stacks improve top-five recall over fixed-session query-aware drilldown on 5 of 6 tasks and reduce median group fragmentation from 285.0 to 157.5 groups, but fixed-session often finds the first positive with less work. Operation stacks also save work against flat summaries on all tasks, while flat remains the full-recall counterpoint and dataset-native hierarchy can reach high recall at high inspection work. The supporting R330–R334 and R355 audits keep this tradeoff honest rather than adding separate experiments: 10,000 paired bootstrap resamples show stable AP, work, budget-recall, and group-count deltas; 2,000 label-permutation trials show AP exceeds the 95% prevalence/group-size null on 6/6 tasks; the 30% work-budget curve reaches median recall 0.3900 versus 0.3559 for fixed-session and 0.0000 for flat; and the fragmentation audit uses fewer groups on 5/6 tasks at the same budget while preserving fixed-session first-positive-work counterpoints. R355 then repeats the comparison at dataset-provided oracle depths: operation-stack query-aware improves fixed-session budget-30 positive-unit recall on 20/24 task-depth rows and uses fewer groups to reach 50% positive units on 22/24. Together, these slices support a faithful localization surface with better inspection-work and fragmentation tradeoffs than flat or fixed-session alone; they do not support a single best hierarchy, a human-productivity claim, or dominance over every trace-tree-shaped objective.

4.3 RQ3/E3: Mechanism and Actionability

Evidence contract: claim, the profiler exposes actionable knobs in stack fields, mapping/tagging rules, rankers, profile specs, and boundary-derived operation fields; oracle, hidden labels used only after profiling, plus held-out OSWorld-Human boundary positives for R358; baseline, default/patched specs, visible rankers, transfer policies, learned-boundary, fixed-session, and semantic-width stacks; metric, accepted patches, AP delta, top-five lift, first-positive-work, group reduction, and top-five work counterpoint; counterpoint, OSWorld-Human needs boundary fields, so this is not automatic boundary discovery, not automatic patch selection, and not a universal selector. Claim-test: RQ3/E3 asks which mechanisms explain the localization gains, and whether profile-guided changes to stack fields, rank features, mappings, or boundary fields improve the output without leaking hidden labels into profiler inputs.

R320 turns localization scores into an optimization diagnosis rather than only a leaderboard, summarized in Table 9. SATraj unsafe is the cleanest operation-stack win: environment, phase, and action fields plus query-aware risk ranking give the best visible top-five F1. Looping exposes the value and limit of repeat_signal: it helps ranking, but positives are prevalent, so prevalence-aware ranking remains the optimization. Side-effect has large oracle headroom and points to deeper write/input mappings. AgentNet step-quality tasks need action-family localization followed by fixed-session examples. OSWorld-Human needs boundary-derived fields rather than action depth alone. Thus the first actionability result is not that one view wins; it is that the profiler identifies which stack field, mapping, ranker, or drilldown should change for each task family.

The Rust mechanism ablations isolate where those diagnoses come from. Stack-text rules and rank-mode variants (R322/R323) are replayable from profile specs, but they are too coarse for safety and side-effect tasks. Operation-level rank features (R324) close that gap by matching visible mapped operation tokens before folding: semantic-stack feature ranking improves AP over width on 5 of 6 tasks, top-five lift on 4 of 6, and first-positive work on 5 of 6; coarse depth improves AP on 4 of 6 while reducing groups. Leave-one-feature and robustness ablations (R325/R326) make the knobs concrete: repeat_signal, write-action features, and status=success are critical in different families, while misleading safety-like or input-phase features can hurt AP or first-positive work. Equal/global feature banks often preserve the gains, and R325-guided repairs improve AP on 2 of 3 misleading-feature cases, but these repairs use offline scored feedback and are not an automatic learned ranker.

The view and policy audits show the same mechanism at the analysis-surface level. Best AP and best top-five F1 are split across operation-stack, fixed-session, dataset-native, raw-action, and flat views; leave-task source selection cannot reliably pick the best visible view. Nevertheless, the operation-stack query-aware policy remains Pareto-frontier on all six tasks, beats flat top-five work on all six, beats fixed-session top-five recall on 5 of 6, and reaches fewer groups-to-50% positives than fixed-session on 5 of 6. At a 25% recall target it covers all six tasks with median work 0.2000 versus flat 1.0000 and median 16.0 groups versus fixed-session 50.0; at 50% recall, other depths or views become the best-work counterpoints. Sequence and oracle-depth audits add the inspection unit below and above individual groups. R355 shows that at a 30% operation budget, operation-stack query-aware reaches median 0.4669

Table 8: R373 task-level claim verdict. Each row combines E2 localization/work evidence with E3 actionability evidence from tracked artifacts; R373 does not rerun the profiler or create data.

Task	Fidelity/work evidence	Fixed-tree tradeoff	Actionability	Verdict
Looping	AP +0.189 vs flat; top-5 work 0.494.	R@5 delta 0.413; groups 40 vs 29. fixed-session finds first positive with less work.	R354 accept_patch: AP 0.7653->0.8808, top-5 lift delta 0.2820	supports-with-counterpoints
Side effect	AP +0.121 vs flat; top-5 work 0.145.	R@5 delta 0.114; groups 40 vs 29. fixed-session has fewer groups.	R354 accept_patch: AP 0.3661->0.4252, top-5 lift delta 0.1822	supports-with-counterpoints
Unsafe action	AP +0.342 vs flat; top-5 work 0.042.	R@5 delta 0.204; groups 142 vs 250. fixed-session finds first positive with less work.	R354 accept_patch: AP 0.1000->0.6081, top-5 lift delta 2.2936	supports-with-counterpoints
Incorrect step	AP +0.029 vs flat; top-5 work 0.001.	R@5 delta -0.009; groups 289 vs 835. fixed-session remains a drilldown baseline.	R354 accept_patch: AP 0.0687->0.0847, top-5 lift delta 2.5010	supports-with-counterpoints
Redundant step	AP +0.015 vs flat; top-5 work 0.009.	R@5 delta 0.005; groups 260 vs 549. fixed-session finds first positive with less work.	R354 accept_patch: AP 0.0863->0.0874, top-5 lift delta 0.8679	supports-with-counterpoints
Human group start	AP +0.035 vs flat; top-5 work 0.407.	R@5 delta 0.353; groups 173 vs 320. fixed-session finds first positive with less work.	R354 rejects visible rank patch; R358 boundary fields improve AP 0.2402->0.2583 and groups 108->74	supports-with-counterpoints

positive-session recall versus fixed-session 0.3230, and across 24 oracle-depth rows it has median 0.4342 budget-30 positive-unit recall, beats fixed-session unit recall on 20/24 rows, and reaches 50% positive units with fewer groups on 22/24. Positive-run proxy units are derived episode units, not human intent boundaries. R355's fixed-session depth-gap counterpoint remains, so this is depth-aware triage, not complete intent-boundary recovery. R344 also keeps the metric surface honest: it records 30 supporting baseline-metric comparisons, 16 explicit counterpoints, and nDCG cases where flat or dataset-native views can win.

The actionability portfolio turns those comparisons into reviewer-auditable profile-configuration tuning insight. R335/R341 show that all 36 objective rows have a concrete configuration action, but 27/36 best visible policies are not the default operation-stack query-aware view; transfer misses are mostly view or ranker changes rather than opaque errors. R345/R346/R347 present the same evidence as diagnostic lenses, case packets, and baseline contrasts: six lenses cover 36 objectives, top-five case groups contain positives on 6/6 tasks, top-one groups on 5/6, median top-five lift is 1.6508 at 0.0937 work, and operation stacks beat flat top-five work on 6/6 and fixed-session top-five recall on 5/6 while preserving fixed-session first-positive and flat full-recall counterpoints. R348/R349 then separate visible counterfactual actions from target-label leakage: 36/36 best rows are visible non-oracle, 27/36 require a non-default action, 25/36 require a view change, and held-out action transfer stays within tolerance on 35/60 aligned decisions but exactly transfers the action class only 7 of 60 times and only 2 of 42 times on non-default target actions. R350 packages this into bounded evidence packets: top-five positives on 6/6 tasks, top-one positives on 5/6, strict 30% work budget on 4/6, and the same transfer guardrail. The actionable claim is therefore a tunable diagnosis surface, not a label-free automatic selector.

Finally, executable patches close one loop from diagnosis to profiler configuration. R354 writes default and profile-guided Rust profile specs over the same visible operation input and scores hidden labels only after agentpprof -profile-spec. Five of six patches improve AP, top-five lift, and first-positive work, with median AP delta 0.0376 and median top-five lift delta 0.5750. The rejected OSWorld-Human patch is the important negative case: visible phase/action rank features are insufficient for a human-boundary objective. R358 follows that diagnosis by folding held-out boundary-derived operation fields; learned-boundary profiles improve AP to 0.2583 versus 0.2402 for semantic width and reduce groups to 74 versus 108, while top-five work and first-positive work increase. This supports

boundary fields as an actionable operation-field knob, not automatic boundary discovery or automatic patch selection. The result belongs to mechanism/actionability, not a fifth core experiment.

4.4 RQ4/E4: Reproducibility and Artifact Hygiene

Evidence contract: claim, the offline profiler path is replayable over tracked inputs at low local cost; oracle, tracked specs/inputs, repeated profile outputs, runtime logs, and source-status rows; baseline, default output versus deterministic-output replay, semantic profile hashes versus raw-byte profile hashes; metric, deterministic spec pass rate, profiler invocations, median/p95 runtime, sample equality, stack equality, and raw-byte output equality; counterpoint, not live eBPF overhead, not human utility, not complete ecosystem compatibility, and not a universal selector. Claim-integrity and reviewer-style checks are artifact hygiene, not empirical evidence. Claim-test: RQ4/E4 asks whether reviewers can replay the offline profile-spec path and verify that source provenance, paper numbers, two-abstraction wording, and non-claims remain aligned.

R327/R328 are the empirical content of this subsection. They rerun the existing profile-spec artifacts used by the labeled-trace experiments: 76 tracked specs over tracked operation JSONL inputs, executed twice for 152 profiler invocations. The semantic profile content is stable for 76/76 specs in both modes; deterministic output also makes raw-byte profile files stable for 76/76 specs by fixing profile timestamps. The clean deterministic rerun reports median runtime 1.601s and p95 2.767s per spec. This supports replayable offline profiling artifacts, not live eBPF overhead or analyst productivity.

The remaining paper gates are artifact hygiene, not empirical evidence. They check that table generation, claim wording, source provenance, non-claims, and the operation/operation-stack boundary remain aligned with tracked outputs. We keep those checks in the artifact ledger so reviewers can audit consistency, but they do not increase the localization, actionability, or overhead claims. The paper-facing summary of their role is Table 3: evidence generators, visualization portfolios, headline selectors, and reviewer gates may support E1-E4, but they do not create additional main experiments.

Profile-spec determinism and local cost. R327 reruns the existing profile-spec artifacts used by R300, R324, and R326. The workload contains 76 tracked specs: four view specs, 12 operation-feature rank specs, and 60 robustness specs. Each spec is executed twice by the Rust profiler against tracked operation JSONL inputs, for 152

Table 9: E3 actionability summary. Each row names the profiler-selected visible policy and the corresponding profile-configuration change suggested by the hidden-label localization results.

Task	Best visible policy	Optimization insight
Looping	dataset-native:query-aware	Keep repeat-signal fields but add prevalence-aware ranking because positives are common.
Side-effect	fixed-session:width	Increase write/input weights or use a deeper side-effect mapping before ranking.
Unsafe operation	operation-stack:query-aware	Use environment, phase, and action stack fields and prioritize risky write actions.
Incorrect step	raw-action:width	Use desktop action family first, then drill into fixed sessions for examples.
Redundant step	raw-action:width	Raw action/status currently beats mapped stack depth, so tune quality-specific fields.
Group start	flat:width	Add boundary-derived fields because action-depth stacks fragment starts.

Table 10: R327 profile-spec reproducibility and offline execution cost. Runtime is seconds per spec, measured after the Rust binary exists. Semantic hashes exclude the JSON generated_at field; raw-byte hashes are reported separately.

Workload	Specs	Med. s	P95 s	Med. stacks
R300 view specs	4	3.448	4.273	631.0
R324 rank-feature specs	12	1.539	2.692	41.0
R326 robustness specs	60	1.542	2.640	41.0
All specs	76	1.581	2.719	42.0

profiler invocations. No dataset is downloaded or synced. The determinism gate hashes the semantic profile content after removing the JSON generated_at timestamp, and separately reports raw-byte hashes.

All 76 specs reproduce the same semantic profile hash, sample count, and unique-stack count across repetitions. Raw-byte determinism is only 4 of 76, because JSON outputs intentionally include a generation timestamp; the stable semantic hash keeps the reproducibility claim on the profile content. Median output size is 37,546 bytes, the largest output is 306,099 bytes, and the largest profile has 2,012 unique stacks. This supports an offline artifact and profile-spec reproducibility claim, not live eBPF overhead or human utility.

R328 closes the raw-artifact caveat with an explicit deterministic-output mode. The Rust CLI accepts `-deterministic-output`, and profile specs may set `deterministic_output`; the mode replaces JSON generated_at and pprof profile time with fixed values while leaving profile content unchanged. R328 reruns the same 76 specs and 152 invocations with this flag. Both semantic and raw-byte determinism are 76 of 76. The clean rerun records empty git and code status. The median runtime in this run is 1,601.0827 ms and the p95 is 2,767.1653 ms; we report these as artifact costs for this run, not as a performance comparison with R327.

5 Related Work and Novelty

Classic profiling and trace analysis. Flame graphs and pprof establish the folded-stack lineage for profiling; profiles consist of weighted samples attached to a location hierarchy, can carry sample labels, and pprof can visualize labels as pseudo stack frames through tag-root or tag-leaf options [1, 2]. Perfetto generalizes trace

Table 11: R327/R328 determinism checks over the same 76 profile specs. R328 uses `-deterministic-output`.

Run	Mode	Sem.	Raw	Med. s	P95 s
R327	default	76/76	4/76	1.581	2.719
R328	deterministic	76/76	76/76	1.601	2.767

ingestion and SQL analysis over many trace formats and can derive new events from trace contents [3]. These systems are closer than a simple “fixed hierarchy” baseline. AGENTPPROF therefore does not claim novelty for query-time aggregation alone. The difference is the domain object and evaluation target: the sample is an agent operation, the frames are recursive multi-field operation projections, and the groups are scored against hidden agent-trajectory labels across datasets.

Agent tracing and LLM observability. OpenTelemetry GenAI and OpenInference standardize GenAI spans, LLM calls, agent reasoning steps, tool invocations, retrieval operations, MCP, and provider-specific telemetry [4, 5]. LangSmith, Langfuse, and Phoenix expose traces, sessions, observations, evaluations, dashboards, datasets, and prompt iteration workflows [6–8]. AgentOps gives the closest academic same-problem framing by arguing that agent observability should trace goals, plans, tools, artifacts, and associated data [9]. These systems are strong prior work, so the safe novelty claim is not generic “agent observability.” In this paper, traces, spans, sessions, and observations are exchange containers or baseline shapes. R320 evaluates fixed-session drilldown as the current trace-tree-shaped baseline; real OpenTelemetry/OpenInference or Phoenix span-tree imports remain future interoperability baselines. The profiler abstractions are only operations and operation stacks, and the central mechanism is that stack shape determines semantic aggregation at query time.

Agent failure and span localization. Recent agent-diagnosis work is the closest same-problem threat. AgentRx releases failed trajectories with critical-failure-step and failure-category annotations and uses constraint checking plus an LLM judge to localize root causes [22]. TELBench/DRIFT builds a deep-research span-localization benchmark from semantic spans annotated for harmful errors [23]. Holistic Evaluation and Failure Diagnosis pairs agent-level diagnosis with per-span evaluation over long traces [24]. AgentAtlas argues that outcome leaderboards miss trajectory and

control-decision quality [25]. These works make clear that failure localization and trajectory diagnosis are not novel by themselves. The narrower difference here is that AGENTPPROF treats profile groups as profiler outputs: the same operations are folded into flat, fixed-session drilldown, dataset-native, raw-action, and operation-stack views, then scored with AP, nDCG, recall under inspection budgets, work-to-first-positive, fragmentation, and profile-configuration actionability across heterogeneous public labeled traces. AgentRx- or TELBench-style traces are therefore future oracle/baseline candidates, not current evidence for a broader span-localization claim.

Public labeled agent trajectories. AgentRewardBench, OSWorld-Human, OpenCUA/AgentNet, SATraj-OS, and OSWorld provide the labels and workloads that make R320 possible [12, 18–21]. Prior benchmark papers use these labels to evaluate agents, reward models, safety behavior, or efficiency. We use them as hidden oracles for a profiler benchmark: different stack-shaped aggregates become ranked outputs, and the evaluation measures precision, recall, AP, nDCG, inspection work, fragmentation, and profile-configuration tuning insight. This is the paper’s scoped novelty.

6 Discussion

The result scope is deliberately narrow. The main claim is a profiler claim: on six real labeled tasks, operation-stack profiles can be scored as ranked localization outputs against dataset-provided labels, and they expose a work/fragmentation/actionability trade-off against flat summaries, fixed-session drilldown, dataset-native hierarchy, and raw-action stacks. The supporting audits add uncertainty checks, label-permutation negative controls, view/depth task-fit checks, fixed-recall target costs, sequence-scope scoring, oracle-depth scoring, action counterfactuals, transfer guardrails, and reproducibility gates. They do not add a new empirical claim; they explain when the headline comparison should be trusted and when it should be narrowed.

The strongest negative results define the useful boundary. Operation-stack query-aware ranking improves AP over width on all six R320 tasks, but top-five F1, first-positive work, and high-recall targets remain task dependent. Mapping helps some tasks and hurts others; feature ablations find both critical and misleading fields; held-out action transfer is often within tolerance but rarely transfers the exact action class for non-default targets. Fixed-session drilldown remains a real first-positive counterpoint, raw-action stacks can cover broader session scope, and OSWorld-Human needs boundary-derived fields. These counterpoints are why the design exposes stack fields, mapping rules, rank modes, operation-level rank features, profile specs, and task-specific policies instead of hard-coding one hierarchy.

These results do not show that humans or agents answer questions faster with the tool, nor do they measure live eBPF overhead. They also do not show automatic discovery of all intent boundaries, a label-free automatic selector, or complete compatibility with OpenTelemetry, Phoenix, LangSmith, Langfuse, or Perfetto producers. Those systems remain related work and baselines. In this paper, trace formats are containers; fixed sessions are the evaluated drilldown baseline, while real span-tree imports remain future baselines. The profiler abstractions remain only operations and operation stacks.

7 Conclusion

AGENTPPROF reduces agent profiling to two abstractions. Operations carry all observed and derived fields; operation stacks fold those fields at the depth chosen by the query. Mapping, predicates, rank rules, profile specs, trace exchange, and boundary backends are mechanisms around those two objects, not additional profiler abstractions.

Across 15 public labeled trace sources, the model covers web, mobile, desktop, software, API, dialogue, failure, safety, human-boundary, and step-quality trajectories. On the four oracle-rich families, operation-stack profiling localizes and ranks labeled problems with less inspection work than flat summaries and a better median-fragmentation tradeoff than fixed-session drilldown, while the mechanism audits show which fields, mappings, rankers, depths, and profile-spec patches produce the gains. The paper’s central claim is therefore a profiler claim: operation/operation-stack profiling exposes dataset-labeled task-relevant failures, quality problems, and semantic boundaries, and produces profile-configuration actionability without relying on prompt/session/span boundaries.

The evidence remains scoped. The profile-spec and deterministic-output audits make the offline artifacts reproducible; the claim-integrity and rubric gates keep the paper aligned with tracked results. They do not claim human or agent analyst productivity, live eBPF overhead, automatic discovery of all intent boundaries, a label-free universal selector, or complete compatibility with OpenTelemetry, Phoenix, LangSmith, Langfuse, or Perfetto producers.

References

- [1] Brendan Gregg. Flame Graphs. <https://www.brendangregg.com/flamegraphs.html>.
- [2] Google pprof documentation. <https://github.com/google/pprof/blob/main/doc/README.md>.
- [3] Perfetto Trace Processor documentation. <https://perfetto.dev/docs/analysis/trace-processor>.
- [4] OpenTelemetry GenAI semantic conventions. <https://github.com/open-telemetry/semantic-conventions-genai>.
- [5] OpenInference specification. <https://arize-ai.github.io/openinference/spec/>.
- [6] LangSmith Observability. <https://docs.langchain.com/langsmith/observability>.
- [7] Langfuse Observability. <https://langfuse.com/docs/observability/overview>.
- [8] Arize Phoenix documentation. <https://arize.com/docs/phoenix>.
- [9] Liming Dong, Qinghua Lu, and Liming Zhu. AgentOps: Enabling Observability of LLM Agents. <https://arxiv.org/abs/2411.05285>.
- [10] McGill-NLP WebLINX. <https://huggingface.co/datasets/McGill-NLP/WebLINX>.
- [11] OpenGVLab GUI-Odyssey. <https://huggingface.co/datasets/OpenGVLab/GUI-Odyssey>.
- [12] WukLab OSWorld-Human. <https://github.com/WukLab/osworld-human>.
- [13] xlangai AgentNet. <https://huggingface.co/datasets/xlangai/AgentNet>.
- [14] McGill-NLP AgentRewardBench. <https://huggingface.co/datasets/McGill-NLP/agent-reward-bench>.
- [15] AI45Research SATraj-OS. <https://huggingface.co/datasets/AI45Research/SATraj-OS>.
- [16] AgentSuite tau-bench trajectories. <https://huggingface.co/datasets/AgentSuite/tau-bench-trajectories>.
- [17] OpenGVLab ScaleCUA-Data. <https://huggingface.co/datasets/OpenGVLab/ScaleCUA-Data>.
- [18] Xing Han Lù et al. AgentRewardBench: Evaluating Automatic Evaluations of Web Agent Trajectories. <https://arxiv.org/abs/2504.08942>.
- [19] Tianbao Xie et al. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. <https://arxiv.org/abs/2404.07972>.
- [20] Xinyuan Wang et al. OpenCUA: Open Foundations for Computer-Use Agents. <https://arxiv.org/abs/2508.09123>.
- [21] Shanghai AI Lab. Safactory: A Scalable Agentic Infrastructure for Training Trustworthy Autonomous Intelligence. <https://arxiv.org/abs/2605.06230>.
- [22] Shraddha Barke et al. AgentRx: Diagnosing AI Agent Failures from Execution Trajectories. <https://arxiv.org/abs/2602.02475>.
- [23] NJU-LINK. Where Do Deep-Research Agents Go Wrong? Span-Level Error Localization in Agent Trajectories. <https://arxiv.org/abs/2606.02060>.

Conference'17, July 2017, Washington, DC, USA

[24] Holistic Evaluation and Failure Diagnosis of AI Agents. <https://arxiv.org/abs/2605.14865>.

[25] Parsa Mazaheri and Kasra Mazaheri. AgentAtlas: Beyond Outcome Leaderboards for LLM Agents. <https://arxiv.org/abs/2605.20530>.